<Coding𝄞onata />

4 Ways to Use HttpClient in ASP.NET Core

Aram Tchekrekjian

@AramT87

Introduction

Are you still doing *new HttpClient()*?

The problem is that you will get issues like Socket exhaustion, difficulty in testing, and no support for Polly (resilience).

Introduction

There are better, cleaner, and safer ways to do that in ASP.NET Core

Keep reading to learn about the 4 ways to use HttpClient properly

And the last one is my favorite.

1

IHttpClientFactory

IHttpClientFactory

The most basic but better alternative for HttpClient

You can use IHttpClientFactory directly

Great for flexible setups to create clients on demand.

Use it when you need dynamic control over the client without the need for strong abstraction

@AramT87

IHttpClientFactory

```
using System.Net.Http;
using System.Threading.Tasks;

public interface ITestApiService
{
    Task<string> GetDataAsync();
}

public class TestApiService(IHttpClientFactory httpClientFactory)
    : ITestApiService
{
    public async Task<string> GetDataAsync()
    {
        var client = httpClientFactory.CreateClient();

        var response = await client.GetAsync("https://httpbin.org/get");

        if (!response.IsSuccessStatusCode)
        {
            throw new HttpRequestException("Failed to get data from test API.");
        }

        return await response.Content.ReadAsStringAsync();
    }
}

// Program.cs

builder.Services.AddHttpClient();
builder.Services.AddScoped<ITestApiService, TestApiService>();
```

@AramT87

2

Named HttpClient

Named HttpClient

Good for working with multiple APIs, each with different configs.

Use when you want centralized configuration but don't want to define a class per API.

You can retrieve your HttpClient by its name when calling the factory.createClient

Named HttpClient

```
using System.Net.Http;
using System.Threading.Tasks;

public interface ITestApiService
{
    Task<string> GetDataAsync();
}

public class TestApiService(IHttpClientFactory httpClientFactory)
    : ITestApiService
{
    public async Task<string> GetDataAsync()
    {
        var client = httpClientFactory.CreateClient("TestApi");

        var response = await client.GetAsync("get");

        if (!response.IsSuccessStatusCode)
        {
            throw new HttpRequestException("Failed to get data from test API.");
        }

        return await response.Content.ReadAsStringAsync();
    }
}

// Program.cs

builder.Services.AddHttpClient("TestApi", client =>
{
    client.BaseAddress = new Uri("https://httpbin.org/");
});
builder.Services.AddScoped<ITestApiService, TestApiService>();
```

3

Typed HttpClient

Typed HttpClient

With typed client, you can directly inject the HttpClient in your service or endpoint, whenever you want to use it.

Best for testability and clean DI

No need to use the factory directly.

Typed HttpClient

```
using System.Net.Http;
using System.Threading.Tasks;

public interface ITestApiService
{
    Task<string> GetDataAsync();
}

public class TestApiService(HttpClient httpClient) : ITestApiService
{
    public async Task<string> GetDataAsync()
    {
        var response = await httpClient.GetAsync("get");

        if (!response.IsSuccessStatusCode)
        {
            throw new HttpRequestException("Failed to get data from test API.");
        }

        return await response.Content.ReadAsStringAsync();
    }
}

// Program.cs

builder.Services.AddHttpClient<ITestApiService, TestApiService>
    ("TestApi", client =>
{
    client.BaseAddress = new Uri("https://httpbin.org/");
});
```

4

Refit

@AramT87

Refit

Refit is an automatic type-safe REST library in .NET

It allows you to handle and manage API calls in a clean and declarative way

It leverages the interfaces and attributes at its core that would translate to RESTful APIs.

Refit uses dynamic proxy generation to build an implementation of the interface at runtime

Refit

Refit.HttpClientFactory by glennawatson, reactiveui, **64.9M** downloads 8.0.0
Refit HTTP Client Factory Extensions

```
using Refit;

public interface ITestApi
{
    [Get("/get")]
    Task<ApiResponse<string>> GetDataAsync();
}

public interface ITestApiService
{
    Task<string> GetDataAsync();
}

public class TestApiService(ITestApi testApi)
    : ITestApiService
{
    public async Task<string> GetDataAsync()
    {
        var response = await testApi.GetDataAsync();

        if (!response.IsSuccessStatusCode)
        {
            throw new HttpRequestException("Failed to get data from test API.");
        }

        return response.Content ?? "No data.";
    }
}

// Program.cs
builder.Services
    .AddRefitClient<ITestApi>()
    .ConfigureHttpClient(c =>
        c.BaseAddress = new Uri("https://httpbin.org/")
    );
builder.Services.AddScoped<ITestApiService, TestApiService>();
```

@AramT87

Found this useful?

Consider Reposting

Thank You

Follow me for more content

Aram Tchekrekjian

Microsoft® MVP Most Valuable Professional

AramT87

Get Free tips and Tutorials in .NET and C#

Join 1,200+ Readers

CodingSonata.com/newsletters

<CodingSonata />